Lecture 4: Embeddings & Transformer Re-rankers

Information Retrieval — Lecture Notes

From Bag-of-Words to Contextualized Representations

Avishek Anand
Information Retrieval Course

February 12, 2026

Abstract

These lecture notes accompany Lecture 4 of the Information Retrieval course, covering the transition from sparse, term-based representations to **dense, neural representations** for document ranking. We develop the conceptual progression from static word embeddings (Word2Vec, GloVe) through the **transformer architecture** to **BERT-based cross-encoder re-rankers**, and show how these compose with BM25 in the standard **two-stage retrieve-then-rerank pipeline**. We also cover the practical aspects of **training cross-encoders** on the MS MARCO dataset, including negative sampling, batching, and common mistakes, as well as **evaluation methodology** using TREC Deep Learning Track benchmarks. An introduction to **ColBERT** (late interaction) previews the dense retrieval methods of Lecture 5. The notes assume familiarity with BM25 and language models from Lecture 3.

Contents

1 From Sparse to Dense: Why We Need Embeddings

In Lecture 3, we established BM25 and statistical language models as the workhorses of term-based retrieval. These methods are fast, well-understood, and remarkably effective—but they share a fundamental limitation: they operate on *exact term matching*. The query "car problems" and the document "automobile issues" have zero term overlap, so BM25 assigns a score of zero regardless of the obvious semantic relationship. Query expansion (RM3, LLM-based methods) partially addresses this, but the underlying representation remains a sparse, high-dimensional vector where each dimension corresponds to a single vocabulary term.

This lecture introduces *dense representations*: low-dimensional vectors where each dimension encodes a learned, latent semantic feature. In this space, "car" and "automobile" are nearby points, and relevance can be measured by vector similarity.

1.1 The Representation Gap

The contrast between sparse and dense representations is stark:

	Sparse (BM25/TF-IDF)	**Dense (Embeddings)**
Dimensionality	$\|V\| \approx 30{,}000$	$d \approx 768$
Non-zero entries	$< 0.1\%$	100%
Each dimension	One vocabulary term	Latent semantic feature
Matching	Exact (lexical)	Approximate (semantic)

The question is: how do we learn dense representations that capture word meaning?

1.2 The Distributional Hypothesis

The theoretical foundation for word embeddings is the *distributional hypothesis*, attributed to Firth [**?**]:

> "You shall know a word by the company it keeps."

The idea is simple but powerful: words that appear in similar contexts tend to have similar meanings. "Cat" and "dog" both appear near "pet," "food," "veterinarian," and "cute," so they should have similar representations. "Cat" and "algorithm" rarely share context, so they should be distant.

Example 1.1 (Unknown word inference)**.** Consider the made-up word "tezguino." From the sentences "A bottle of *tezguino* is on the table," "Everybody enjoys *tezguino*," "*Tezguino* makes you drunk," and "We brew *tezguino* from corn," we can infer—without a dictionary—that tezguino is an alcoholic beverage. We know the word purely from the company it keeps.

1.3 Word2Vec: Words as Vectors

Mikolov et al. [**?**] operationalized the distributional hypothesis by training a shallow neural network to predict context words from a target word (Skip-gram) or a target word from context (CBOW). The key insight is that the learned weight matrix of this network *is* the word embedding.

Definition 1.1 (Word2Vec (Skip-gram))**.** Given a vocabulary V and an embedding dimension d, the Skip-gram model learns two matrices: an input embedding $W_{\text{in}} \in \mathbb{R}^{|V| \times d}$ and an output

embedding $W_{\text{out}} \in \mathbb{R}^{|V| \times d}$. For each word w in a training corpus, the model maximizes the probability of observing the words in a context window around w:

$$\max_{\theta} \sum_{(w,c) \in \text{data}} \log P(c \mid w; \theta) = \log \frac{\exp(\vec{w}_{\text{in}} \cdot \vec{c}_{\text{out}})}{\sum_{v \in V} \exp(\vec{w}_{\text{in}} \cdot \vec{v}_{\text{out}})} \tag{1}$$

After training, the input embeddings W_{in} are used as the word vectors.

In practice, the softmax over the full vocabulary is approximated using negative sampling [**?**] or hierarchical softmax, making training tractable on large corpora (billions of words).

Famous property: linear analogies. Word2Vec embeddings encode relational structure as vector directions:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}} \tag{2}$$

Gender, tense, plurality, and other linguistic properties are captured as directions in the embedding space. For IR, the relevant consequence is that semantically similar words ("car" and "automobile") have high cosine similarity: $\cos(\vec{\text{car}}, \vec{\text{automobile}}) \approx 0.7$.

1.4 GloVe: Global Vectors

Pennington et al. [**?**] proposed GloVe (Global Vectors for Word Representation), an alternative that directly factorizes the word co-occurrence matrix rather than using local context windows. The objective is:

$$\min_{W, \tilde{W}} \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\vec{w}_i \cdot \tilde{\vec{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \tag{3}$$

where X_{ij} is the co-occurrence count between words i and j, f is a weighting function that downweights very frequent co-occurrences, and b_i, $\tilde{b}_j$ are bias terms.

In practice, Word2Vec and GloVe produce embeddings of similar quality. Both are available pre-trained on large web corpora (Common Crawl, Wikipedia) in dimensions ranging from 50 to 300.

1.5 From Words to Documents: The Naïve Approach

The simplest way to represent a document as a dense vector is to average its word embeddings:

$$\vec{d} = \frac{1}{|d|} \sum_{w \in d} \vec{w} \tag{4}$$

This produces a fixed-size vector for any document, and documents about similar topics will be nearby in the embedding space. However, the approach has severe limitations:

- **Word order is lost**: "dog bites man" and "man bites dog" have identical representations.
- **Polysemy is unresolved**: "bank" (financial) and "bank" (river) have the same vector, regardless of context.
- **Important words are drowned out**: Rare, informative terms are averaged with frequent, uninformative ones.

These limitations motivate the need for *contextualized* embeddings, where the representation of each word depends on the surrounding sentence.

1.6 Static vs. Contextualized Embeddings

The distinction between static and contextualized embeddings is one of the most important conceptual shifts in modern NLP:

Static embeddings (Word2Vec, GloVe): Each word *type* has exactly one vector, regardless of context. "Bank" always maps to the same point in the embedding space, whether it appears in "river bank" or "bank account."

Contextualized embeddings (ELMo, BERT, GPT): Each word *token* gets a different vector depending on the surrounding sentence. "Bank" in "I deposited money at the bank" receives a different representation than "bank" in "We had a picnic by the river bank."

Contextualized embeddings solve both the synonymy problem ("car" and "automobile" get similar vectors in similar contexts) and the polysemy problem ("bank" gets different vectors in different contexts). The key technology that enables contextualized embeddings is the *transformer architecture*.

Key Takeaway

The progression is: bag-of-words (no semantics) → static embeddings (word-level semantics, no context) → contextualized embeddings (full semantic understanding). Each step unlocks new capabilities for IR.

2 The Transformer Architecture

The transformer, introduced by Vaswani et al. [**?**] in the landmark paper "Attention Is All You Need," is the architecture underlying BERT, GPT, and virtually all modern language models. Understanding the transformer is essential for understanding neural ranking.

2.1 Motivation: Why Not RNNs?

Before transformers (pre-2017), the dominant architectures for processing text were recurrent neural networks (RNNs) and their variants (LSTMs, GRUs). These models process words sequentially—one at a time, left to right—and maintain a hidden state that carries information from previous words.

RNNs have two critical limitations:

1. **Sequential bottleneck**: Each time step depends on the previous one, making training inherently sequential and slow. Parallelization across words is impossible.
2. **Long-range dependencies**: Information from early words must pass through many intermediate states to reach later words, suffering from vanishing (or exploding) gradients. In practice, RNNs struggle with dependencies spanning more than 50–100 words.

The transformer addresses both issues by replacing recurrence with *self-attention*: every word attends to every other word directly, in parallel. There is no sequential bottleneck, and long-range dependencies are captured in a single step.

2.2 Self-Attention: The Core Mechanism

Definition 2.1 (Scaled Dot-Product Attention)**.** Given a sequence of n tokens, each represented as a d-dimensional vector, self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{5}$$

where $Q \in \mathbb{R}^{n \times d_k}$ (queries), $K \in \mathbb{R}^{n \times d_k}$ (keys), and $V \in \mathbb{R}^{n \times d_v}$ (values) are linear projections of the input.

The intuition behind the three projections is:

- **Query** ($\vec{q}_i$): "What am I looking for?" — what information does token i need from other tokens?
- **Key** ($\vec{k}_j$): "What do I contain?" — what information does token j advertise to other tokens?
- **Value** ($\vec{v}_j$): "What information do I provide?" — the actual content that token j contributes.

The attention computation proceeds in four steps:

1. **Dot products**: Compute $\vec{q}_i \cdot \vec{k}_j$ for all pairs (i, j). This measures how relevant token j is to token i.
2. **Scaling**: Divide by $\sqrt{d_k}$ to prevent the dot products from becoming very large (which would push the softmax into saturation, producing near-binary attention weights with vanishing gradients).
3. **Softmax**: Normalize to produce attention weights α_{ij} that sum to 1 across j.
4. **Weighted sum**: The output for token i is $\sum_j \alpha_{ij} \vec{v}_j$, a weighted combination of all tokens' values.

Example 2.1 (Self-attention for coreference)**.** In the sentence "The animal didn't cross the street because **it** was too tired," self-attention learns to assign a high weight between "it" and "animal" (because the query of "it" matches the key of "animal"), resolving the coreference. This happens in a single step, regardless of the distance between the two words.

Example 2.2 (Worked attention computation)**.** Consider 2 tokens ("retrieval," "models") with $d_k = 2$:

	retrieval	models
$\vec{q}$	$[1.0, 0.5]$	$[0.3, 1.2]$
$\vec{k}$	$[0.8, 0.2]$	$[0.1, 1.0]$
$\vec{v}$	$[0.3, 0.9]$	$[0.7, 0.4]$

Computing output for "retrieval":
Step 1: $\vec{q}_{\text{ret}} \cdot \vec{k}_{\text{ret}} = 1.0 \times 0.8 + 0.5 \times 0.2 = 0.9$; $\quad \vec{q}_{\text{ret}} \cdot \vec{k}_{\text{mod}} = 1.0 \times 0.1 + 0.5 \times 1.0 = 0.6$.
Step 2: Scale by $\sqrt{2} \approx 1.41$: $[0.64, 0.43]$.
Step 3: Softmax: $[\exp(0.64), \exp(0.43)] = [1.90, 1.54]$. Normalize: $[0.55, 0.45]$.
Step 4: $\vec{h}_{\text{ret}} = 0.55 \times [0.3, 0.9] + 0.45 \times [0.7, 0.4] = [0.48, 0.68]$.
The output for "retrieval" now incorporates information from "models"—it is no longer an isolated representation but a context-aware one.

2.3 Multi-Head Attention

A single attention head captures one type of relationship between tokens. But words relate to each other in multiple ways simultaneously: syntactically (subject-verb), semantically (synonymy), positionally (adjacent words), and referentially (coreference). Multi-head attention addresses this by running h independent attention computations in parallel, each with its own learned projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot W^O \tag{6}$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

BERT uses $h = 12$ heads. Analysis of trained heads shows that different heads specialize in different linguistic phenomena: some attend to syntactic structure, others to semantic similarity, others to positional patterns [**?**].

2.4 Positional Encoding

Self-attention is permutation-invariant: if we shuffle the input tokens, the attention weights change but the computation is identical in structure. This means that without additional information, "dog bites man" and "man bites dog" would produce the same representations—word order is lost entirely.

To inject position information, transformers add a *positional encoding* vector to each token embedding before feeding it into the attention layers:

$$\vec{x}_i = \vec{e}_{\text{word}_i} + \vec{p}_i \tag{7}$$

Vaswani et al. used sinusoidal encodings:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d}) \tag{8}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d}) \tag{9}$$

Each position gets a unique pattern, and nearby positions have similar encodings. BERT uses *learned* positional embeddings instead (up to 512 positions), which are parameters trained alongside the rest of the model.

2.5 The Transformer Encoder Block

A single transformer encoder block consists of:

1. Multi-head self-attention
2. Add & layer normalization (residual connection)
3. Position-wise feed-forward network (two linear layers with ReLU)
4. Add & layer normalization (residual connection)

The residual connections (skip connections) are essential: they allow gradients to flow directly from later layers to earlier layers during backpropagation, enabling training of deep networks. Without them, training 12+ layers would be extremely unstable.

BERT-base stacks 12 such blocks, with 768-dimensional hidden states and 12 attention heads, for a total of approximately 110 million parameters.

3 BERT for Information Retrieval

BERT (Bidirectional Encoder Representations from Transformers) was introduced by Devlin et al. [**?**] and rapidly became the foundation for neural approaches to IR. Its key innovation is *bidirectional pre-training*: unlike GPT (which processes text left-to-right), BERT sees the full context in both directions when computing each token's representation.

3.1 Pre-training Objectives

BERT is pre-trained on a large unlabeled corpus (BooksCorpus + English Wikipedia, approximately 3.3 billion words) using two self-supervised objectives:

Masked Language Modeling (MLM): 15% of input tokens are randomly replaced with a `[MASK]` token, and the model predicts the original token from the surrounding context. For example, given "The `[MASK]` sat on the mat," BERT must predict "cat." This forces the model to build deep, bidirectional representations of language.

Next Sentence Prediction (NSP): Given two sentences A and B, the model predicts whether B is the actual next sentence after A or a random sentence. This teaches inter-sentence relationships. (Later work showed that NSP is less important than MLM and can be removed without significant quality loss [**?**].)

Pre-training takes approximately 4 days on 16 TPU chips. The result is a model that "knows language" in a deep, transferable sense: it understands synonymy, polysemy, negation, and compositional semantics.

3.2 Fine-Tuning BERT as a Cross-Encoder

To use BERT for ranking, Nogueira and Cho [**?**] introduced a simple but powerful approach: feed the query and document *jointly* into BERT and use the output of the `[CLS]` token as a relevance signal.

Definition 3.1 (Cross-Encoder)**.** Given a query q and a document d, the cross-encoder computes a relevance score as follows:

1. Concatenate: `[CLS]` q `[SEP]` d `[SEP]`
2. Encode with BERT (12 layers of full cross-attention between all query and document tokens)
3. Extract the final hidden state of the `[CLS]` token: $h_{\texttt{[CLS]}} \in \mathbb{R}^{768}$
4. Apply a linear classification head: $P(\text{relevant}) = \sigma(W \cdot h_{\texttt{[CLS]}} + b)$

The crucial property is that query and document tokens attend to each other *through all 12 layers*. This enables rich interaction: the model can learn that "heart attack" in the query should attend strongly to "myocardial infarction" in the document, that "not relevant" means something different from "relevant," and that the overall topic of the document (not just individual term matches) determines relevance.

3.3 Why Cross-Encoders Are So Accurate

Cross-encoders address all three limitations of bag-of-words methods:

1. **Vocabulary mismatch:** Through cross-attention, "car" in the query attends to "automobile" in the document, and the model learns from training data that this is a strong relevance signal.
2. **Semantic relevance vs. topical overlap:** BM25 ranks any document containing "coffee" highly for the query "is coffee bad for you," including documents about coffee brands or coffee growing regions. A cross-encoder understands the *health* intent and promotes documents about health effects.
3. **Negation and nuance:** The query "countries without death penalty" matches BM25 documents about the death penalty indiscriminately. The cross-encoder understands that "without" reverses the intent and promotes documents listing abolitionist countries.

3.4 Why Cross-Encoders Cannot Retrieve

Despite their accuracy, cross-encoders have a fatal efficiency limitation: they must process every (query, document) pair independently at inference time. There is no way to pre-compute document representations, because the document's representation depends on the query (through cross-attention).

Example 3.1 (Latency analysis)**.** At approximately 5 ms per (query, document) pair on a modern GPU:

Candidates	Total time	Feasible?
1,000,000 (full corpus)	5,000 seconds	No
10,000	50 seconds	No
1,000	5 seconds	Borderline
100	0.5 seconds	Yes
20	0.1 seconds	Yes

The conclusion is clear: cross-encoders can realistically re-rank the top 100–1,000 documents from a first-stage retriever, but they *cannot* replace the retriever itself. This is the fundamental constraint that gives rise to the two-stage pipeline.

Example 3.2 (Cross-encoder scoring)**.** Query: "side effects of aspirin."

Doc A: "Aspirin may cause stomach bleeding and ringing in the ears as common adverse reactions." Doc B: "The company's stock price went up after the aspirin sales report."

Through cross-attention, Doc A's terms "adverse reactions," "bleeding," and "stomach" attend strongly to the query's "side effects." The model assigns $P(\text{rel}) = 0.94$. Doc B's terms "stock price" and "sales report" do not attend to "side effects" at all; the model assigns $P(\text{rel}) = 0.12$.

Note that BM25 might rank Doc B competitively, since it contains "aspirin" with high TF. The cross-encoder correctly identifies that the *topic* does not match the intent.

3.5 Key Cross-Encoder Variants

Several variants of the cross-encoder architecture have been proposed:

monoBERT [?]: The original. Uses BERT-base or BERT-large with a linear classification head on `[CLS]`. 110M–340M parameters. Simple, well-understood, widely available.

monoT5 [?]: Uses T5, a sequence-to-sequence model. The input is "Query: q Document: d Relevant:" and the model generates "true" or "false." The score is $P(\text{"true"})$. The T5-3B variant slightly outperforms monoBERT on most benchmarks but is significantly slower.

RankGPT [?]: A novel approach that gives a large LLM (GPT-4) a *list* of documents and asks it to sort them by relevance. This is *listwise* re-ranking (as opposed to the pointwise approach of monoBERT). No fine-tuning is needed. Extremely expensive but extremely effective. Represents the frontier of LLM-based ranking as of 2024.

Production Note

For most applications, monoBERT (or its distilled variant, DistilBERT) provides the best quality-to-latency trade-off. monoT5-3B is used when accuracy is paramount and latency budget allows 2+ seconds per query. RankGPT is primarily a research tool; its cost ($0.01–0.10 per query) makes it impractical for high-volume applications.

3.6 WordPiece Tokenization

BERT does not tokenize text by whitespace. Instead, it uses WordPiece [**?**], a subword tokenization algorithm that splits rare words into known subwords:

Input	**WordPiece tokens**
"information retrieval"	["information", "retrieval"]
"transformers"	["transform", "##ers"]
"BM25 algorithm"	["bm", "##25", "algorithm"]

The "##" prefix indicates a continuation of the previous token. BERT's vocabulary contains 30,522 WordPiece tokens. The maximum sequence length is 512 tokens; for cross-encoder re-ranking, the query (∼10 tokens) and document (∼500 tokens) must fit within this limit, requiring truncation of long documents.

Common Pitfall

Long documents are truncated at 512 tokens, potentially losing relevant content. For passage-level datasets like MS MARCO, this is rarely an issue (passages are typically 50–200 words). For document-level ranking, strategies include first-passage extraction, sliding windows, and max-passage scoring.

4 ColBERT: Late Interaction

The cross-encoder achieves high accuracy by allowing full interaction between query and document tokens, but this interaction prevents pre-computation of document representations. The bi-encoder (covered in detail in Lecture 5) goes to the opposite extreme: it encodes query and document entirely independently, producing a single vector for each, and computes relevance as a dot product. This enables pre-computation and fast retrieval via approximate nearest neighbor (ANN) search, but sacrifices the fine-grained interaction that makes cross-encoders accurate.

ColBERT [**?**] proposes a middle ground: *late interaction*.

Definition 4.1 (ColBERT)**.** Encode the query and document **separately** with BERT, producing per-token embeddings $\{\vec{q}_1, \ldots, \vec{q}_{|q|}\}$ and $\{\vec{d}_1, \ldots, \vec{d}_{|d|}\}$ (each projected to 128 dimensions). Compute relevance via MaxSim:

$$\text{score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \vec{q}_i \cdot \vec{d}_j \tag{10}$$

For each query token, find the document token with the highest dot-product similarity, then sum these maximum similarities across all query tokens.

The key insight is that documents can be encoded *offline* (since BERT_d does not see the query), and the per-token embeddings can be stored. At query time, only the query needs to be encoded (through BERT_q), and the MaxSim computation is fast.

	Cross-Encoder	**ColBERT**	**Bi-Encoder (L5)**
Interaction	Full (all layers)	Late (token-level)	None (single vector)
Pre-compute docs	No	Yes (per-token)	Yes (single vector)
Use for retrieval	No	Yes (with ANN)	Yes (with ANN)
Accuracy	Highest	Near cross-encoder	Lower
Storage per doc	N/A	$\lvert d\rvert \times 128$ floats	1×768 floats

ColBERT's trade-off is storage: storing per-token embeddings (approximately 100–200 128-dimensional vectors per document) requires significantly more space than a single 768-dimensional vector. Subsequent work (ColBERTv2 [**?**]) introduced residual compression to reduce this overhead by approximately 6–10×.

Key Takeaway

ColBERT represents a continuum between the accuracy of cross-encoders and the efficiency of bi-encoders. It achieves near-cross-encoder accuracy while allowing document pre-computation, at the cost of higher storage. We will revisit ColBERT in detail in Lecture 5.

5 Training Cross-Encoders in Practice

This section covers the practical aspects of training a cross-encoder re-ranker, from dataset selection through negative sampling to hyperparameter tuning. These details are often glossed over in research papers but are critical for reproducing strong results.

5.1 MS MARCO: The Standard Training Dataset

The MS MARCO (Microsoft MAchine Reading COmprehension) passage ranking dataset [**?**] is the standard benchmark for training and evaluating neural ranking models. It consists of:

- **8.8 million passages** extracted from web documents
- **530,000 training queries** with sparse relevance labels
- Approximately **1 relevant passage per query** (sometimes 2–3)
- **6,980 development queries** for validation

Common Pitfall

MS MARCO has *sparse* labels: for each query, only one (or a few) passages are judged relevant out of 8.8 million. This means many passages that are actually relevant are *unlabeled*. The practical consequence is that we cannot treat all unlabeled passages as truly non-relevant—some are false negatives. This is why the negative sampling strategy (discussed below) matters so much.

5.2 Training Pipeline

Training a cross-encoder follows six steps:

1. **Generate training triples**: For each query q, pair the labeled positive passage d^+ with a sampled negative passage d^- to form triples (q, d^+, d^-).
2. **Tokenize**: Convert each (query, passage) pair into WordPiece tokens with the format `[CLS]` q `[SEP]` d `[SEP]`, padded or truncated to 512 tokens.
3. **Batch**: Group tokenized pairs into mini-batches of 16–32 (limited by GPU memory).
4. **Forward pass**: BERT encodes each (query, passage) pair, producing a relevance score $P(\text{rel})$ for each.

5. **Compute loss**: Binary cross-entropy for pointwise training, or a margin-based loss for pairwise training (ensuring $\text{score}(q, d^+) > \text{score}(q, d^-)$).

6. **Backpropagation and update**: Compute gradients and update all parameters using AdamW.

Typical training takes 2–3 epochs over the MS MARCO training triples, approximately 12–24 hours on a single GPU.

5.3 Negative Sampling Strategies

The choice of negative examples is arguably the single most important decision in training a cross-encoder. There are three main strategies, listed in order of increasing effectiveness:

5.3.1 Random Negatives

Sample a passage uniformly at random from the corpus. Since the corpus has 8.8 million passages and only ∼1 is relevant per query, a random passage is almost certainly non-relevant. However, these negatives are *easy*: the model can distinguish a random passage from a relevant one based on topical cues alone, without learning fine-grained relevance signals. Random negatives provide a weak training signal and are insufficient as the sole source of negatives.

5.3.2 BM25 Hard Negatives (Recommended)

Run BM25 on each training query and take the top-k passages that are *not* labeled as relevant. These passages are topically related to the query (they contain the query terms) but are not relevant. This forces the model to learn beyond term overlap—to distinguish between a document that merely mentions "aspirin" and one that actually discusses aspirin's side effects.

BM25 hard negatives are the standard approach and provide approximately 80% of the benefit of more sophisticated methods. They are easy to generate (a single BM25 retrieval pass) and do not require any pre-trained model.

5.3.3 Cross-Encoder Mined Negatives (Advanced)

Use a trained cross-encoder to score all candidate passages for each query, then take the highest-scoring non-relevant passages as negatives. These are the "hardest" negatives—passages that the current model finds most confusing. Retrain the model on these new negatives, then mine again. This iterative process (used in ANCE [**?**]) produces the strongest models but is expensive: each iteration requires scoring millions of passages with a neural model.

Key Takeaway

Start with BM25 hard negatives. They are simple, cheap, and effective. Only move to iterative mining if you need the last 1–2 NDCG points and have the computational budget.

5.4 Batching and GPU Training

Neural re-ranker training is constrained by GPU memory. The key considerations are:

Sequence length: Each (query, passage) pair is tokenized to at most 512 WordPiece tokens. Query tokens are typically ∼10; passage tokens are ∼200–400. Longer sequences consume more memory quadratically (due to the attention matrix).

Batch size: With BERT-base and 512-token sequences, a 24 GB GPU (e.g., RTX 3090, A5000) can fit a batch of 16–32 pairs. Larger batches generally lead to more stable training.

Gradient accumulation: To simulate a larger effective batch size without requiring more memory, accumulate gradients over multiple forward passes before performing a weight update. For example, 4 accumulation steps with batch size 8 yields an effective batch size of 32.

Mixed precision (fp16): Store model weights and activations in 16-bit floating point instead of 32-bit. This approximately halves memory usage and provides a $\sim$1.5$\times$ speedup, with negligible quality loss.

5.5 Hyperparameters

The following hyperparameters are standard for cross-encoder fine-tuning on MS MARCO:

Hyperparameter	Recommended value
Learning rate	1–3×10^{-5}
Optimizer	AdamW ($\epsilon = 10^{-6}$)
Warmup	10% of total training steps
Learning rate schedule	Linear decay after warmup
Epochs	2–3
Batch size (effective)	32–64
Max sequence length	512 tokens
Evaluation frequency	Every 5,000–10,000 steps

5.6 Common Training Mistakes

Common Pitfall

(1) Too many epochs. Cross-encoders overfit quickly on MS MARCO. Beyond 3 epochs, validation performance typically degrades. Monitor dev MRR@10 closely and stop early.
(2) Learning rate too high. Rates above 3×10^{-5} cause *catastrophic forgetting*: the model loses the language understanding acquired during pre-training and performance collapses. The fine-tuning learning rate must be much smaller than the pre-training learning rate.
(3) Only random negatives. Without hard negatives, the model learns to distinguish random passages from relevant ones (trivial) but never learns to distinguish topically related but non-relevant passages from truly relevant ones (the actual task).
(4) Ignoring document truncation. Long passages are silently truncated at 512 tokens. For MS MARCO passages this is rarely an issue, but for longer documents, consider first-passage, sliding-window, or max-passage strategies.

6 Evaluation: Metrics and TREC Deep Learning

Rigorous evaluation is essential for comparing ranking systems. This section covers the metrics, datasets, and best practices used in the neural IR community.

6.1 Evaluation Metrics

6.1.1 MRR@k (Mean Reciprocal Rank)

Definition 6.1 (MRR@k)**.**

$$\text{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank of first relevant document (within top } k)} \tag{11}$$

MRR measures how quickly the system finds *one* relevant document. It is most appropriate for QA-style queries where the user needs a single answer. MRR@10 is the primary metric for MS MARCO passage ranking.

6.1.2 NDCG@k (Normalized Discounted Cumulative Gain)

Definition 6.2 (NDCG@k)**.**

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)} \qquad \text{NDCG@}k = \frac{\text{DCG@}k}{\text{Ideal DCG@}k} \tag{12}$$

where rel_i is the relevance grade of the document at position i, and Ideal DCG is computed by sorting documents by decreasing relevance.

NDCG handles graded relevance (e.g., 0 = not relevant, 1 = marginally relevant, 2 = relevant, 3 = highly relevant). It is position-weighted: relevant documents at higher positions contribute more than the same document lower in the ranking. The normalization ensures the score is in $[0, 1]$.

Example 6.1 (NDCG@5 computation)**.** Ranked results for "best Italian restaurants":

Rank	**Doc**	**Rel.**	**Gain** ($2^{\text{rel}} - 1$)	**Discounted**
1	A	3	7	$7/\log_2(2) = 7.00$
2	B	0	0	0.00
3	C	2	3	$3/\log_2(4) = 1.50$
4	D	1	1	$1/\log_2(5) = 0.43$
5	E	3	7	$7/\log_2(6) = 2.71$
			DCG@5	11.64

Ideal ranking (sorted: 3, 3, 2, 1, 0): Ideal DCG@5 $= 7/1 + 7/1.58 + 3/2 + 1/2.32 + 0 = 13.36$.
NDCG@5 $= 11.64/13.36 =$ **0.87**.

The penalty comes from the non-relevant document at rank 2 and the highly relevant document pushed to rank 5.

6.1.3 MAP (Mean Average Precision)

MAP computes precision at each position where a relevant document appears, averages these precisions for each query, then averages across queries. It is standard for binary relevance (older TREC evaluations) but does not handle graded relevance.

6.2 TREC Deep Learning Track

The TREC Deep Learning Track [**?**], organized by NIST since 2019, is the gold standard for evaluating neural ranking systems. Key properties:

- Uses the MS MARCO passage corpus (8.8M passages).

- 43–76 queries per year with *dense expert judgments*: 100–200 passages per query are judged by trained NIST assessors on a 4-point graded scale (0–3).
- Primary metric: **NDCG@10**.
- Participating systems submit ranked lists; NIST evaluates using the judgments.

Remark 6.1 (Why TREC DL matters)**.** MS MARCO's development set has sparse labels (only ∼1 relevant passage per query), which can be misleading: a system might retrieve a perfectly relevant passage that happens to be unlabeled and receive no credit. TREC DL's dense judgments (100–200 judged passages per query) provide far more reliable evaluation. This is why the community reports results on both: MRR@10 on MS MARCO dev for comparability with the literature, and NDCG@10 on TREC DL for reliable evaluation.

6.3 Evaluation Best Practices

1. **Train on MS MARCO, evaluate on TREC DL.** The MS MARCO training set provides ample labeled data; TREC DL provides reliable evaluation.
2. **Always include BM25 and BM25+RM3 baselines.** BM25 is the universal baseline. Some papers report gains over BM25 that disappear against BM25+RM3—always check against both.
3. **Test statistical significance.** TREC DL has only 43–76 queries. Differences between systems may not be statistically significant. Use a paired t-test or bootstrap test, and apply Bonferroni correction for multiple comparisons.
4. **Report per-query analysis.** Averages hide important information. A system might improve on 30 queries and degrade on 20. Per-query analysis reveals failure modes and helps diagnose the system.
5. **Report latency alongside quality.** A system that achieves 0.02 higher NDCG@10 but takes 10× longer is not necessarily better. Report quality-latency Pareto curves.

7 The Two-Stage Retrieve-then-Rerank Pipeline

The efficiency limitation of cross-encoders and the accuracy limitation of BM25 naturally combine into the **two-stage retrieve-then-rerank pipeline**, which is the de facto architecture of modern search.

7.1 Architecture

1. **Stage 1 — Retrieval (BM25 or Dense):** Retrieve the top-1,000 candidate documents from the full corpus. This stage is *recall-oriented*: its goal is to include as many relevant documents as possible, tolerating some irrelevant ones. BM25 executes in ∼5 ms via the inverted index (Lecture 2). RM3 or LLM expansion (Lecture 3) can improve recall.
2. **Stage 2 — Re-ranking (Cross-Encoder):** Score the top-100 candidates with a BERT cross-encoder and re-order them. This stage is *precision-oriented*: its goal is to push truly relevant documents to the top. At ∼5 ms per candidate, scoring 100 documents takes ∼500 ms.

The pipeline takes ∼505 ms total per query, well within the latency budget of most interactive applications.

7.2 The Recall Ceiling

The most important property of the two-stage pipeline is the **recall ceiling**: the re-ranker can only reorder documents that Stage 1 retrieves. If BM25 misses a relevant document (i.e., it is not in the top-1,000), the cross-encoder can never recover it, no matter how accurate it is.

This has two implications:

1. Stage 1 quality directly bounds overall system quality. Investing in better Stage 1 retrieval (RM3, dense retrieval) pays dividends through the entire pipeline.
2. Evaluating Stage 1 in isolation via Recall@1000 is essential. A system with 0.85 Recall@1000 has a fundamentally higher ceiling than one with 0.70.

7.3 Pipeline Variants and Typical Performance

Pipeline	NDCG@10	Latency	Notes
BM25 only	∼0.45	5 ms	Sparse baseline
BM25 + RM3	∼0.49	15 ms	Classical expansion
BM25 + monoBERT	∼0.55	500 ms	Neural re-ranking
BM25 + RM3 + monoBERT	∼0.57	520 ms	Best sparse + neural
BM25 + monoT5-3B	∼0.58	2 s	Larger model

Approximate figures on MS MARCO passage ranking.

Several observations:

- The cross-encoder adds ∼10 NDCG points over BM25—a very large improvement.
- RM3 still helps even when BERT re-ranks (it improves Stage 1 recall, raising the ceiling).
- Larger models help but with diminishing returns and significantly higher latency.

7.4 What the Cross-Encoder Fixes

A per-query analysis reveals three categories where cross-encoders consistently improve over BM25:

Vocabulary mismatch: Queries using lay terms ("heart attack") now find documents using medical terminology ("myocardial infarction").

Semantic relevance: Documents that are topically related but not relevant (e.g., a coffee brand page for the query "is coffee bad for you") are demoted in favor of documents that actually address the query intent.

Negation and nuance: Queries with negation ("countries *without* death penalty") or complex intent are handled correctly.

Common Pitfall

The cross-encoder *fails* when BM25 does not retrieve the relevant document at all. The most common cause is vocabulary mismatch so severe that no query terms appear in the document. This is the primary motivation for dense retrieval (Lecture 5), which performs semantic matching at the retrieval stage.

8 Hands-On: PyTerrier Experiments

This section provides code for implementing and evaluating the two-stage pipeline using PyTerrier [**?**] and Hugging Face models.

8.1 Setup

```
import pyterrier as pt
if not pt.started():
    pt.init()

dataset = pt.get_dataset('irds:vaswani')
index = pt.IndexFactory.of("./vaswani_index")
topics = dataset.get_topics().head(10)
qrels = dataset.get_qrels()

# BM25 baseline
bm25 = pt.BatchRetrieve(index, wmodel="BM25")

# Load monoT5 re-ranker
from pyterrier_t5 import MonoT5ReRanker
monoT5 = MonoT5ReRanker(model='castorini/monot5-base-msmarco')
```

8.2 BM25 vs. Cross-Encoder

```
# Two-stage pipeline: BM25 retrieves top-100, monoT5 re-ranks
pipeline_rerank = (bm25 % 100) >> monoT5

# Compare
pt.Experiment(
    [bm25, pipeline_rerank],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["BM25", "BM25 + monoT5"]
)
```

8.3 Pipeline Combinations

```
rm3 = pt.rewrite.RM3(index)

pipelines = {
    "BM25":                 bm25,
    "BM25 + RM3":           bm25 >> rm3 >> bm25,
    "BM25 + monoT5":        (bm25 % 100) >> monoT5,
    "BM25 + RM3 + monoT5": (bm25 >> rm3 >> bm25) % 100 >> monoT5,
}

pt.Experiment(
    list(pipelines.values()), topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "recall_100"],
    names=list(pipelines.keys())
)
```

8.4 Effect of Re-rank Depth

```
depths = [10, 20, 50, 100, 200, 500]
pipelines_depth = {
    f"rerank@{d}": (bm25 % d) >> monoT5 for d in depths
}

pt.Experiment(
    [bm25] + list(pipelines_depth.values()),
    topics, qrels,
```

```
    eval_metrics=["ndcg_cut_10"],
    names=["BM25 only"] + list(pipelines_depth.keys())
)
# Observe: diminishing returns after ~100
```

8.5 TREC DL Evaluation

```
# Load TREC DL 2019 for reliable evaluation
dl19 = pt.get_dataset("irds:msmarco-passage/trec-dl-2019")
topics_dl19 = dl19.get_topics()
qrels_dl19 = dl19.get_qrels()

from pyterrier.measures import nDCG, MAP, RR

pt.Experiment(
    [bm25, pipeline_rerank],
    topics_dl19, qrels_dl19,
    eval_metrics=[nDCG@10, nDCG@100, MAP, RR(rel=2)],
    names=["BM25", "BM25 + monoT5"]
)
```

9 Connections and Preview

9.1 The Representation Spectrum

The methods covered across Lectures 3 and 4 form a spectrum of increasing semantic understanding:

Method	Representation	Semantics	Speed
BM25 (L3)	Bag of words	None	Very fast (retrieval)
Word2Vec/GloVe (L4)	Static embedding	Word-level	Fast
Cross-Encoder (L4)	Contextualized	Full	Slow (re-rank only)
ColBERT (L4/L5)	Per-token embedding	Near-full	Moderate
Bi-Encoder (L5)	Single-vector embedding	Moderate	Fast (retrieval)

Each step adds semantic capability but at a cost: static embeddings lose context, cross-encoders lose speed, and bi-encoders lose fine-grained interaction.

9.2 Preview: Lecture 5 — Dense Retrieval

The central limitation identified in this lecture is that cross-encoders cannot retrieve. Lecture 5 addresses this by introducing *bi-encoders*: models that encode queries and documents independently into dense vectors, enabling semantic retrieval over the full corpus via ANN search.

Key topics in Lecture 5:

- Bi-encoder training (contrastive learning, hard negatives)
- DPR (Dense Passage Retrieval) [?]
- ColBERT in depth (late interaction, compression, indexing)
- Hybrid retrieval (BM25 + Dense)
- Connection to ANN indexing from Lecture 2 (IVF+PQ, FAISS)

9.3 Preview: Lecture 6 — Learning to Rank

All scoring functions introduced in Lectures 3–4 produce *features* that can be combined by a learned ranking function:

- BM25 score, Dirichlet LM score (Lecture 3)
- Cross-encoder relevance score, bi-encoder similarity (Lecture 4–5)
- Document length, query length, IDF, overlap count, etc.

Learning to Rank (LTR) trains a model (e.g., LambdaMART) to learn the optimal combination of these features, directly optimizing a ranking metric like NDCG.

9.4 The Complete Modern Retrieval Stack

Putting it all together:

1. **Stage 1:** BM25 retrieves top-1,000 candidates *or* dense retrieval via bi-encoder + ANN (L5)
2. **Stage 1b (optional):** RM3 or LLM expansion improves recall (L3)
3. **Stage 2:** Cross-encoder (BERT) re-ranks top-100 (L4, this lecture)
4. **Stage 3:** Learning to Rank combines all signals (L6)
5. Output: final ranked list → user or LLM (RAG)

10 Summary

This lecture covered the transition from sparse, term-based retrieval to neural ranking:

1. **Dense representations capture semantics.** Word2Vec and GloVe learn word vectors from co-occurrence patterns. Static embeddings handle synonymy but not polysemy. Contextualized embeddings (BERT) handle both.
2. **Transformers enable contextualized understanding.** Self-attention lets every token attend to every other token. Multi-head attention captures multiple relationship types. Positional encoding preserves word order.
3. **Cross-encoders are the most accurate rankers.** BERT cross-encoders process query and document jointly through full cross-attention, enabling vocabulary mismatch resolution, semantic relevance judgment, and negation understanding. But they are too slow for full-corpus retrieval.
4. **ColBERT offers a middle ground.** Late interaction (per-token MaxSim) achieves near-cross-encoder accuracy while allowing document pre-computation, at the cost of higher storage.
5. **Training requires care.** BM25 hard negatives, low learning rates, 2–3 epochs, and proper evaluation on TREC DL are essential for reproducing strong results.
6. **The two-stage pipeline is the standard architecture.** BM25 (recall) → Cross-encoder (precision). RM3 can further improve Stage 1. This is how modern search engines work.
7. **Evaluation matters.** Train on MS MARCO, evaluate on TREC DL. Report MRR@10 and NDCG@10. Always include BM25 baselines. Test statistical significance.

References

[1] J.R. Firth. A synopsis of linguistic theory 1930–1955. In *Studies in Linguistic Analysis*, pages 1–32. Blackwell, 1957.

[2] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. In *Proceedings of ICLR Workshop*, 2013.

[3] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. Distributed representations of words and phrases and their compositionality. In *Proceedings of NeurIPS*, pages 3111–3119, 2013.

[4] J. Pennington, R. Socher, and C.D. Manning. GloVe: Global vectors for word representation. In *Proceedings of EMNLP*, pages 1532–1543, 2014.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Proceedings of NeurIPS*, pages 5998–6008, 2017.

[6] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[7] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[8] R. Nogueira and K. Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.

[9] R. Nogueira, Z. Jiang, R. Pradeep, and J. Lin. Document ranking with a pretrained sequence-to-sequence model. In *Findings of EMNLP*, pages 708–718, 2020.

[10] W. Sun, L. Yan, X. Ma, P. Ren, D. Yin, and Z. Ren. Is ChatGPT good at search? Investigating large language models as re-ranking agents. In *Proceedings of EMNLP*, pages 14918–14937, 2023.

[11] O. Khattab and M. Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of SIGIR*, pages 39–48, 2020.

[12] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of NAACL*, pages 3715–3734, 2022.

[13] K. Clark, U. Khandelwal, O. Levy, and C.D. Manning. What does BERT look at? An analysis of BERT's attention. In *Proceedings of ACL BlackboxNLP Workshop*, pages 276–286, 2019.

[14] Y. Wu, M. Schuster, Z. Chen, Q.V. Le, M. Norouzi, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[15] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng. MS MARCO: A human generated machine reading comprehension dataset. In *Proceedings of NeurIPS Workshop on CoCo*, 2016.

[16] N. Craswell, B. Mitra, E. Yilmaz, D. Campos, and E.M. Voorhees. Overview of the TREC 2019 deep learning track. In *Proceedings of TREC*, 2020.

[17] L. Xiong, Y. Xiong, D. Li, K. Tang, J. Liu, P. Bennett, J. Ahmed, and A. Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *Proceedings of ICLR*, 2021.

[18] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020.

[19] C. Macdonald, N. Tonellotto, S. MacAvaney, and I. Ounis. PyTerrier: Declarative experimentation in Python from BM25 to dense retrieval. In *Proceedings of CIKM*, pages 4526–4533, 2021.